PoC Engineering TODO Tracker

# HELIX

Engineering TODO Tracker

Extracted from PoC Roadmap v2.0

| | | |
|---|---|---|
| [ ] | **Not started** – Task pending | **To** |
| [x] | **Completed** – Task finished | |
| [~] | **In progress** – Currently working on it | |
| [!] | **Blocked** – Needs review or unblocking | |

**update:** Open `HELIX_TODO.tex` and replace the marker command name (e.g., `\undone` → `\done`).

## 1 PHASE 0 – Project Setup, Git Workflow & Dev Environment

### 1.1 0.1 – Git Repository Setup

[x] Create private repository on GitHub/GitLab

[x] Clone locally into project root

[x] Create `.gitignore`

[ ] Create `CHANGELOG.md`

[ ] Initial commit: `git commit -m "chore: initialize HELIX project"`

[ ] Create `dev` branch: `git checkout -b dev`

### 1.2 0.2 – Apache Vhost Configuration

[x] Install Apache, PHP, and dependencies

[x] Create `/etc/httpd/conf.d/helix.conf`

[x] Add `127.0.0.1 helix.local` to `/etc/hosts`

[x] Fix file permissions

[x] Verify mod_rewrite

[x] Run `sudo httpd -t` (syntax check)

[x] Run `sudo systemctl restart httpd`

[x] Verify `http://helix.local` serves frontend

[x] Create `api/index.php` test file

[x] Verify `http://helix.local/api` returns "API OK"

[x] Check error logs if anything fails

### 1.3 0.3 – MySQL Setup

[x] Start MySQL in XAMPP

[x] Create database `helix_db`

[x] Create dedicated user `helix_user` with limited privileges

[x] Verify connection via phpMyAdmin

### 1.4 0.4 – PHP Composer Setup

[x] Run `composer install`

[x] Verify `vendor/autoload.php` was created

[ ] Add `require_once` for autoloader in `api/index.php`

### 1.5 0.5 – Environment Configuration

[x] Create `api/config/Environment.php`

[x] Create `api/config/.env` (gitignored, with real values)

[x] Create `api/config/.env.example` (committed, no secrets)

### 1.6 0.6 – Database Config

[x] Create `api/config/Database.php`

[ ] Test `Database::getInstance()` in `api/index.php`

### 1.7 0.7 – CORS Configuration

[x] Create `api/config/Cors.php`

### 1.8 0.8 – Helper Classes

[ ] Create `api/helpers/Response.php`

[ ] Create `api/helpers/Validator.php`

### 1.9 0.9 – Front Controller Bootstrap

[ ] Finalize `api/index.php` with routing skeleton

[ ] Verify `http://helix.local/api/nonexistent` returns 404 JSON

### 1.10 0.10 – Python Environment

[x] Confirm Python ≥ 3.10

[x] Create `.venv` inside `python-module/`

[ ] Create `pyproject.toml`

[ ] Install: `pip install -e ".[dev]"`

[ ] Update `PYTHON_PATH` in `.env`

### 1.11 0.11 – Java Environment

[x] Confirm JDK ≥ 17

[ ] Review `java-module/compile.sh`

[ ] Run `compile.sh`

[ ] Verify `.class` files in `java-module/build/`

### 1.12 Phase 0 Validation

[ ] `http://helix.local` → serves `frontend/index.html`

[ ] `http://helix.local/api` → JSON response

[ ] `http://helix.local/api/nonexistent` → 404 JSON

[ ] `composer dump-autoload` runs without errors

[ ] Python: `python -c "import sklearn; print('OK')"` inside `.venv`

[ ] Java: `javac` compiles all source files

[ ] Git log shows meaningful commits

## 2 PHASE 1 – Architecture Design, DB Schema & UML

### 2.1 1.1 – Database Schema

[x] Verify schema matches UML diagrams

[ ] Import: `mysql -u helixdb_admin -p helixdb < db/schema.sql`

[ ] Confirm all 6 tables in phpMyAdmin

### 2.2 1.2 – API Route Contract

[ ] Document all 17 routes in `docs/SDD/API_REFERENCE.md`

[ ] Add every route to `api/index.php` as empty stubs

### 2.3 1.3 – UML Diagram Validation

[ ] Review `use_case.puml` – all 7 actors, correct stereotypes

[ ] Review `er_diagram.puml` – matches schema.sql exactly

[ ] Review `class_services.puml` – Controller → Service → Model

[ ] Review `sequence_auth.puml` – full auth flow

[ ] Review `sequence_log_ingest.puml` – pipeline flow

[ ] Review `sequence_ai_chat.puml` – AI flow

[ ] Review `component.puml` – architecture overview

[ ] Review `activity_alert.puml` – alert lifecycle

[ ] Export all diagrams as PNG to `docs/UML/png/`

### 2.4 1.4 – SRS Traceability Matrix

[ ] Create `docs/traceability_matrix.md`

[ ] Map every FR-001 to FR-063 to route, controller, DB table

### 2.5 1.5 – Seed File

[ ] Generate real bcrypt hashes for test users

[ ] Create `database/seed.sql` with 3 test users

[ ] Import seed data

### 2.6 Phase 1 Validation

[ ] Schema imports without error

[ ] All 6 tables visible in phpMyAdmin

[ ] All UML diagrams reviewed and corrected

[ ] All FR-001 to FR-063 in traceability matrix

[ ] All routes registered as stubs in `api/index.php`

[ ] Git commit: `docs: finalize schema, API contract, UML and traceability matrix`

## 3 PHASE 2 – Authentication Module (FR-001 to FR-007)

### 3.1 2.1 – Auth Middleware

[ ] Create `api/middleware/AuthMiddleware.php`

[ ] Create `api/middleware/RoleMiddleware.php`

### 3.2 2.2 – User Model

[ ] Create `api/models/User.php` with all 5 static methods

### 3.3 2.3 – Auth Service

[ ] Create `api/services/AuthService.php` with register, login, logout, currentUser

### 3.4 2.4 – Auth Controller

[ ] Create `api/controllers/AuthController.php` with register, login, logout, me

### 3.5 2.5 – Register Auth Routes

[ ] Add 4 auth routes to `api/index.php`

### 3.6 2.6 – Frontend: Login Page

[ ] Build `frontend/login.html` with form

[ ] Create `frontend/js/auth.js`

[ ] Create `frontend/js/api.js` shared client

### 3.7 2.7 – Auth Tests

[ ] Create `tests/php/AuthServiceTest.php`

### 3.8 2.8 – Manual Test Checklist

[ ] POST `/api/auth/register` valid data → 201

[ ] POST `/api/auth/register` duplicate username → 409

[ ] POST `/api/auth/register` invalid email → 422

[ ] POST `/api/auth/login` valid credentials → 200

[ ] POST `/api/auth/login` wrong password → 401

[ ] GET `/api/auth/me` with cookie → 200

[ ] GET `/api/auth/me` without cookie → 401

[ ] POST `/api/auth/logout` → 200

[ ] GET `/api/auth/me` after logout → 401

[ ] Test role-restricted route as learner → 403

### 3.9 Phase 2 Validation

[ ] FR-001 to FR-007 checked in traceability matrix

[ ] Passwords stored as bcrypt (hash starts with `$2y$`)

[ ] Wrong password returns same error as non-existent user

[ ] `session_regenerate_id(true)` called on every login

[ ] Frontend redirects to login on 401

[ ] Git commit: `feat(auth): implement authentication, RBAC middleware, login frontend`

## 4 PHASE 3 – Dashboard Module (FR-010 to FR-014)

---

### 4.1 3.1 – Client-Side Router

[ ] Create `frontend/js/router.js`

### 4.2 3.2 – Dashboard Shell

[ ] Create `frontend/dashboard.html`

[ ] Create `frontend/js/ui.js` (role-aware sidebar)

### 4.3 3.3 – Dashboard Stats Endpoint

[ ] Create `api/controllers/DashboardController.php`

[ ] Register `GET /dashboard/stats` route

### 4.4 3.4 – Dashboard Frontend Module

[ ] Create `frontend/js/dashboard.js` with Chart.js rendering

[ ] Include Chart.js CDN in `dashboard.html`

### 4.5 Phase 3 Validation

[ ] FR-010 to FR-014 checked in traceability matrix

[ ] Sidebar shows only role-appropriate links

[ ] Dashboard renders with zero data (no crashes)

[ ] Chart.js charts render with empty/zero datasets

[ ] Client-side routing works (back/forward buttons)

[ ] Git commit: `feat(dashboard): implement dashboard shell, stats API, Chart.js`

## 5 PHASE 4 – Log Ingestion Module (FR-020 to FR-023)

- [ ] Create `api/models/LogEntry.php`
- [ ] Create `api/services/LogService.php`
- [ ] Create `api/controllers/LogController.php` (upload, index, show)
- [ ] Register 3 log routes in `api/index.php`
- [ ] Create `api/services/JavaBridge.php`
- [ ] Build `frontend/js/logs.js`
- [ ] Build log upload UI in frontend
- [ ] Test file upload → parsed → stored in DB
- [ ] FR-020 to FR-023 checked in traceability matrix
- [ ] Git commit: `feat(logs): implement log ingestion pipeline`

## 6 PHASE 5 – Alert Module (FR-030 to FR-034)

- [ ] Create `api/models/Alert.php`
- [ ] Create `api/services/AlertService.php`
- [ ] Create `api/controllers/AlertController.php` (index, show, updateStatus)
- [ ] Register 3 alert routes in `api/index.php`
- [ ] Build `frontend/js/alerts.js`
- [ ] Build alert list + detail UI
- [ ] Implement status transition (open → investigating → resolved)
- [ ] Test alert CRUD operations
- [ ] FR-030 to FR-034 checked in traceability matrix
- [ ] Git commit: `feat(alerts): implement alert management module`

## 7 PHASE 6 – ML Detection Module (FR-040 to FR-044)

- [ ] Create `api/models/AnomalyScore.php`
- [ ] Create `api/services/MLService.php`
- [ ] Create `api/controllers/MLController.php` (score, index)
- [ ] Register 2 ML routes in `api/index.php`

- [ ] Create `python-module/anomaly_detector.py`
- [ ] Implement Isolation Forest scoring in Python
- [ ] Implement PHP ↔ Python bridge via `exec()`
- [ ] Build `frontend/js/ml.js` for score visualization
- [ ] Test: log entry → Python scoring → anomaly_scores table
- [ ] FR-040 to FR-044 checked in traceability matrix
- [ ] Git commit: `feat(ml): implement anomaly detection pipeline`

## 8 PHASE 7 – AI Assistant Module (FR-050 to FR-053)

- [ ] Create `api/models/ChatSession.php`
- [ ] Create `api/services/AIService.php`
- [ ] Create `api/controllers/AIController.php` (chat, history)
- [ ] Register 2 AI routes in `api/index.php`
- [ ] Implement OpenAI API integration in AIService
- [ ] Implement prompt engineering with alert context
- [ ] Implement consent check for Red Team suggestions
- [ ] Build `frontend/js/chat.js`
- [ ] Build AI chat UI with conversation history
- [ ] Test: alert context → AI response → stored in chat_sessions
- [ ] FR-050 to FR-053 checked in traceability matrix
- [ ] Git commit: `feat(ai): implement AI assistant with OpenAI integration`

## 9 PHASE 8 – Scanner Module (FR-060 to FR-062)

- [ ] Create `api/models/ScanResult.php`
- [ ] Create `api/services/ScannerService.php`
- [ ] Create `api/controllers/ScannerController.php` (scan, results)
- [ ] Register 2 scanner routes in `api/index.php`
- [ ] Implement target scope validation

- [ ] Implement legal consent check
- [ ] Implement async nmap execution via `exec()`
- [ ] Implement XML output parsing
- [ ] Build `frontend/js/scanner.js`
- [ ] Build scanner UI with progress tracking
- [ ] Test: scan request → nmap → results stored
- [ ] FR-060 to FR-062 checked in traceability matrix
- [ ] Git commit: `feat(scanner): implement red team reconnaissance module`

## 10 PHASE 9 – Integration & Testing

- [ ] End-to-end test: login → upload logs → ML scoring → alerts → AI analysis
- [ ] End-to-end test: login → run nmap scan → view results
- [ ] Role-based access testing for all 5 roles
- [ ] Security audit: SQL injection, XSS, CSRF checks
- [ ] Performance test: large log file ingestion
- [ ] Error handling: all API error paths return proper status codes
- [ ] Frontend: all pages handle empty states gracefully
- [ ] Documentation: update `README.md` with setup instructions
- [ ] Git commit: `test: integration testing and bug fixes`

## 11 PHASE 10 – Docker & Deployment

- [ ] Create `Dockerfile` for PHP API
- [ ] Create `Dockerfile` for Python ML module
- [ ] Create `docker-compose.yml` (Apache + MySQL + PHP + Python)
- [ ] Configure Apache vhost for container environment
- [ ] Test full stack in Docker
- [ ] Write deployment guide
- [ ] Git commit: `chore: dockerize HELIX platform`

| Phase | Total | Done | In Progress | Blocked |
|---|---|---|---|---|
| Phase 0: Setup | 29 | 16 | 0 | 0 |
| Phase 1: Design | 12 | 0 | 0 | 0 |
| Phase 2: Auth | 19 | 0 | 0 | 0 |
| Phase 3: Dashboard | 8 | 0 | 0 | 0 |
| Phase 4: Logs | 11 | 0 | 0 | 0 |
| Phase 5: Alerts | 11 | 0 | 0 | 0 |
| Phase 6: ML | 12 | 0 | 0 | 0 |
| Phase 7: AI | 13 | 0 | 0 | 0 |
| Phase 8: Scanner | 13 | 0 | 0 | 0 |
| Phase 9: Testing | 9 | 0 | 0 | 0 |
| Phase 10: Docker | 7 | 0 | 0 | 0 |
| **Total** | **144** | **16** | **0** | **0** |